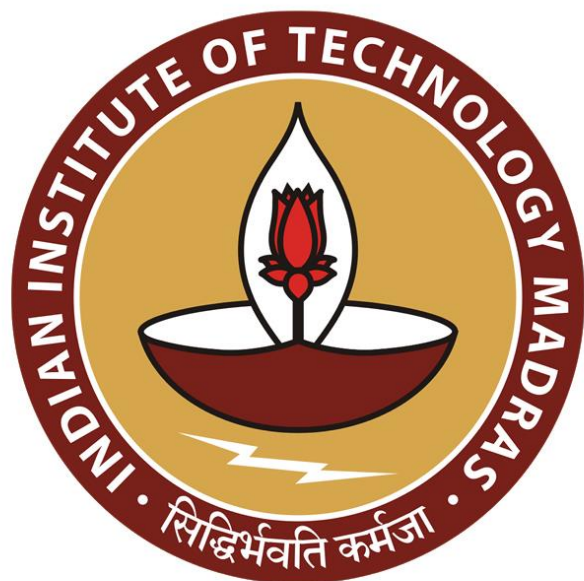


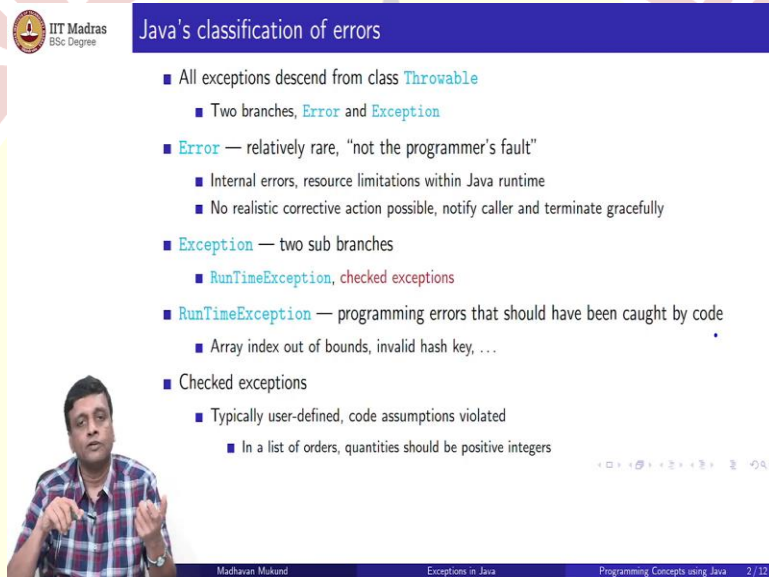


IIT Madras
ONLINE DEGREE

Programming Concepts Using Java
Professor Madhavan Mukund
Department of Computer Science
Chennai Mathematical Institute
Indian Institute of Technology, Madras
Exceptions in Java

So, we have seen what errors and exceptions are in general. So, let us look more closely at how exceptions are handled in the context of Java.

(Refer Slide Time: 0:23)



Java's classification of errors

- All exceptions descend from class `Throwable`
 - Two branches, `Error` and `Exception`
- `Error` — relatively rare, “not the programmer’s fault”
 - Internal errors, resource limitations within Java runtime
 - No realistic corrective action possible, notify caller and terminate gracefully
- `Exception` — two sub branches
 - `RuntimeException`, checked exceptions
- `RuntimeException` — programming errors that should have been caught by code
 - Array index out of bounds, invalid hash key, ...
- Checked exceptions
 - Typically user-defined, code assumptions violated
 - In a list of orders, quantities should be positive integers

Madhavan Mukund Exceptions in Java Programming Concepts using Java 2/12

So, remember that all exceptions are treated as objects. And they all descend from this class called `Throwable`, which splits into two sub classes, the subclass `Error`, which consists of these JVM related issues, these are issues with the resources and all that if the underlying runtime, which you really have no control over. So, in some sense, this is not the programmers fault. And the other side, you have exception.

So, two subclasses, that error and exception. Under exception again you have two groups. The first group is called runtime exception and runtime exception corresponds to errors which occur in the code because the values are kind of globally wrong. In some sense, you are trying to access an array, which is outside the index range, or you are trying to access a key in a hash table, which is not present.

So, these are things ideally in your code, you should be defensive as they say you should say, if i is less than 0 or i is beyond the array length minus 1 then you should not try to access it, but other flag there. So, ideally, these runtime exceptions should be caught by your code and not by the underlying system and flagged as an error. On the other side, you could actually have these checked exceptions which are peculiar to the logic of your code. You do not want to store negative numbers in the list, then whenever a negative number comes, you can say this is illegal and give a suitable error message. So, now what we want to see is how we do all this in Java.

(Refer Slide Time: 1:40)



Catching and handling exceptions

- **try-catch**
 - Enclose code that may generate exception in a **try** block
 - Exception handler in **catch** block
 - Similar to Python

```
try {  
    ...  
    call a function that may  
    throw an exception  
    ...  
} catch (ExceptionType e){  
    ...  
    examine e and handle it  
    ...  
}
```

Diagram: A blue bracket on the right side of the code block groups the try and catch blocks, with the word "handler" written next to it.



Madhavan Mukund Exceptions in Java Programming Concepts using Java 3/12



Catching and handling exceptions

- **try-catch**
 - Enclose code that may generate exception in a **try** block
 - Exception handler in **catch** block
 - Similar to Python
- If **try** encounters an exception, rest of the code in the block is skipped

```
try {  
    ...  
    call a function that may  
    throw an exception  
    ...  
} catch (ExceptionType e){  
    ...  
    examine e and handle it  
    ...  
}
```

Diagram: A blue bracket on the right side of the code block groups the try and catch blocks, with a double-headed vertical arrow indicating the flow between them.



Madhavan Mukund Exceptions in Java Programming Concepts using Java 3/12

try-catch

- Enclose code that may generate exception in a **try** block
- Exception handler in **catch** block
- Similar to Python
- If **try** encounters an exception, rest of the code in the block is skipped
- If exception matches the type in **catch**, handler code executes
- Otherwise, **uncaught** exception is passed back to the code that called this code
- Top level uncaught exception — program crash

```
try {
    ...
    call a function that may
    throw an exception
    ...
}
catch (ExceptionType e){
    ...
    examine e and handle it
    ...
}
```



Madhavan Mukund

Exceptions in Java

Programming Concepts using Java

3/12

So, at the first level, let us look at the side of calling code which has headers. So, this is there in Python also. So, you have this try statement. So, whenever you are going to call code, which potentially might create an error, you call it with this try. So, there is this try block. And in the try block, you put the call to the function, which may throw an exception. And then if an exception actually occurs, it will abort the try block and look for a catch block which matches it.

So, here we have a catch block, which goes with the try block. And this is the handler. So, this is the code that deals with the exception. So, it gets it is like a function in some sense, it gets the exception that was caught, in some sense by the try, which was generated by the try code, and then it tries to do something it. So, you can you can examine that exception and do something to handle it. So, Python also has a similar structure. So, this should be reasonably familiar to you.

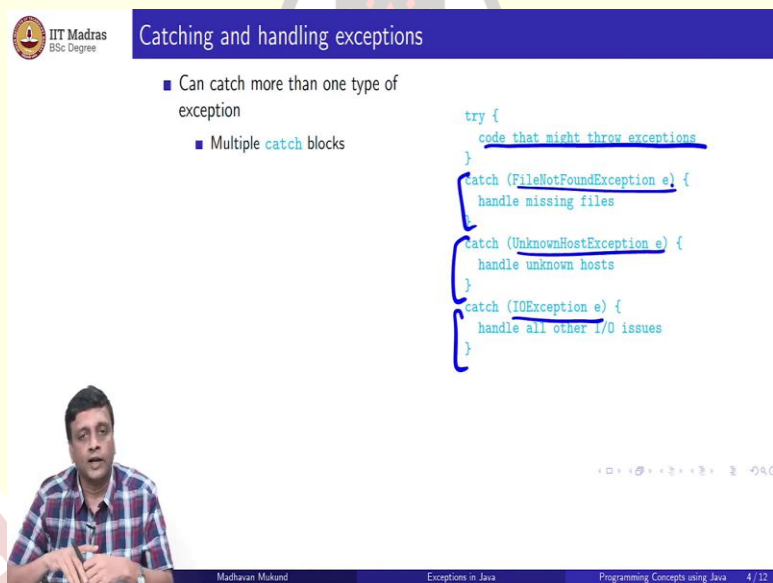
So, the usual thing happens, which is that if try encounters an exception, then whatever. So, if the exception happens here, then these statements beyond that are not executed, you just keep that block and move to the corresponding catch. Hopefully, there is a catch. So, if you match, this is the exception that you got. So, notice that this has a type the catch is now looking for an exception, which belongs to a particular type.

So, if you match the type, then the corresponding block is triggered. Otherwise, if there is no matching catch then this exception is as though it happened without a try. So, you did not you were trying to catch an exception, but not this one, you had no awareness of this one. So, it is as

though you did not try to catch it at all. So, it gets passed back to the caller. The function that called you now gets the exception that you generated by calling a third function.

So, f calls g calls h, h generates an error, it passes back to g, g does not know what to do with it, because it does not catch it explicitly, it goes back to f. And as we said, if this happens all the way up, like f does not know what to do, it passes back to e and so, on. At some point, we started the computation, typically with our main. So, then eventually the error goes back all the way to main, and main has no way to handle it. Then the program will crash and the user of the program will see an error message on the screen saying something bad happened. So, this is the extreme case of what happens when these errors happen. But ideally, we would like to catch it and take some action against it.

(Refer Slide Time: 4:14)



Catching and handling exceptions

- Can catch more than one type of exception
 - Multiple `catch` blocks

```
try {  
    code that might throw exceptions  
}  
catch (FileNotFoundException e) {  
    handle missing files  
}  
catch (UnknownHostException e) {  
    handle unknown hosts  
}  
catch (IOException e) {  
    handle all other I/O issues  
}
```

Madhavan Mukund Exceptions in Java Programming Concepts using Java 4 / 12

Catching and handling exceptions

- Can catch more than one type of exception
 - Multiple `catch` blocks
- Exceptions are classes in the Java class hierarchy
 - `catch (ExceptionType e)` matches any subtype of `ExceptionType`

```
try {
    code that might throw exceptions
}
catch (FileNotFoundException e) {
    handle missing files
}
catch (UnknownHostException e) {
    handle unknown hosts
}
catch (IOException e) {
    handle all other I/O issues
}
```



Madhavan Mukund

Exceptions in Java

Programming Concepts using Java 4/12

Catching and handling exceptions

- Can catch more than one type of exception
 - Multiple `catch` blocks
- Exceptions are classes in the Java class hierarchy
 - `catch (ExceptionType e)` matches any subtype of `ExceptionType`
- Catch blocks are tried in sequence
 - Match exception type against each one in turn

```
try {
    code that might throw exceptions
}
catch (FileNotFoundException e) {
    handle missing files
}
catch (UnknownHostException e) {
    handle unknown hosts
}
catch (IOException e) {
    handle all other I/O issues
}
```



Madhavan Mukund

Exceptions in Java

Programming Concepts using Java 4/12

सिद्धिर्भवति कर्मजा

- Can catch more than one type of exception
 - Multiple `catch` blocks
- Exceptions are classes in the Java class hierarchy
 - `catch (ExceptionType e)` matches any subtype of `ExceptionType`
- Catch blocks are tried in sequence
 - Match exception type against each one in turn
- Order `catch` blocks by argument type, more specific to less specific
 - `IOException` would intercept `FileNotFoundException`

```
try {
    code that might throw exceptions
} catch (FileNotFoundException e) {
    handle missing files
} catch (UnknownHostException e) {
    handle unknown hosts
} catch (IOException e) {
    handle all other I/O issues
}
```

Subclass (handwritten note pointing from IOException to FileNotFoundException)



Madhavan Mukund

Exceptions in Java

Programming Concepts using Java

4/12

So, what we saw is that we want to match the exception and catch it, but there may be multiple types of errors. So, the good thing is that you can have multiple catch associated one try because this code might generate many different types of exceptions. And you might have to take different corrective action depending on the type of exception. So, you have several catch blocks here. And each of them catches a different type of exception.

So, this is like our function signature. So, we have the same name, all of them are called catch, but they have different argument types of based on the argument type the appropriate one is chosen. So, just like in other functions. This argument type is compatible with the hierarchy. So, if you have something which is of a mean if you have a catch, which whose argument is accept expected to be of some class exception type.

Then there may be many different subclasses of that type any one of them will work. So, in Java for instance IO exception is actually a built-in exception class and file not found exception is a particular case of an IO exception. So, if you are expecting an IO exception, it is to get a file not found exception did the usual thing, if you are expecting the superclass, you are okay to get an object of the subclass there is no type error. So, that is exactly what happens in catch.

So, then we have to disambiguate as to which catch is actually going to be used because the same argument may match different argument means the same exception that comes may match different document types, because of this compatibility of this hierarchy with the object

hierarchy. So, what happens in Java and even in Python is that these catch blocks are actually tried in sequence.

So, you first try to see if `e` matches `file not found`. If it does not match `file not found`, then you check whether it is an `unknown host exception`. If it does not match `unknown host exception`, then you catch check for an `IO exception`. So, it goes in sequence, and the first one that matches the one that is executed. Now, in particular, this means that you must make sure that you must go from more specific, less specific.

So, as I said, `file not found exception` is a subclass of `IO exception`. So, supposing this one had come on top. Then if I actually got a `file not found exception`, it would match `IO exception` and the `IO exception` code would trigger. So, it would not reach the code which is specifically meant for `file not found exception`. So, you must go from more specific sub classes to higher super classes. So, in some sense, as you go down, it becomes more and more generic, the kind of exception that you are trying to handle.

(Refer Slide Time: 7:04)

Generating exceptions

- When does a function generate an exception?
- **Error** — JVM runtime issue
- **RuntimeException**
 - Array index out of bounds, invalid hash key, ...
- Code calls another function that generates an exception
- Your code detects an error and generates an exception
 - **throw** a checked exception

Madhavan Mukund Exceptions in Java Programming Concepts using Java 5/12

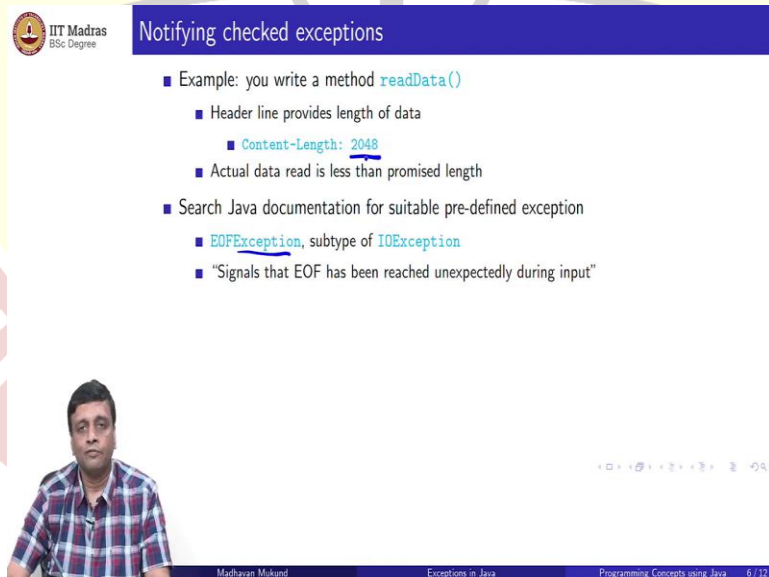
So, let us look from the other side now. So, we saw what we will come back to this thing in more detail. But we saw how you would catch an exception if the code that you are trying to execute generates an exception. So, how does code generate an exception. So, as we said before, there is this exception error of type an exception of type capital E error, which is a Java runtime issue.

And this is something which happens because of some resource issues within the runtime and you cannot do much about it then you have a runtime exception.

And runtime exception, we say that typically, issues with coding, like you try to access an array out of bounds and so, on. And this is something that ideally, your code should try to check and catch. Now, you might call another function which generates an exception and you do not handle it. So, then your code generates an exception. So, you are not generating the exception in your code directly, but indirectly, you call f calls g, g generates an error.

So, implicitly, f generates an error, because f does not have a try catch for it. And the fourth case is when you actually detect something anomalous something wrong happening in your code, and you choose to generate an exception. So, this is the checked exception case where you want to throw something explicitly. So, these are the typical situations in which a function might generate an exception either it happens because that runtime something went wrong or it called a function which generated an exception and did not catch it, or it voluntarily decided to throw a checked exception.

(Refer Slide Time: 8:34)



 IIT Madras BSc Degree

Notifying checked exceptions

- Example: you write a method `readData()`
 - Header line provides length of data
 - `Content-Length: 2048`
 - Actual data read is less than promised length
- Search Java documentation for suitable pre-defined exception
 - `EOFException`, subtype of `IOException`
 - "Signals that EOF has been reached unexpectedly during input"

 Madhavan Mukund

Exceptions in Java Programming Concepts using Java 6/12

Notifying checked exceptions

- Example: you write a method `readData()`
 - Header line provides length of data
 - `Content-Length: 2048`
 - Actual data read is less than promised length
- Search Java documentation for suitable pre-defined exception
 - `EOFException`, subtype of `IOException`
 - "Signals that EOF has been reached unexpectedly during input"
- Create an object of exception type and `throw` it

```
throw new EOFException();
```

EOFException e = new
EOFException();



Madhavan Mukund

Exceptions in Java

Programming Concepts using Java 6/12

Notifying checked exceptions

- Example: you write a method `readData()`
 - Header line provides length of data
 - `Content-Length: 2048`
 - Actual data read is less than promised length
- Search Java documentation for suitable pre-defined exception
 - `EOFException`, subtype of `IOException`
 - "Signals that EOF has been reached unexpectedly during input"
- Create an object of exception type and `throw` it

```
throw new EOFException();
```

- Can also pass a diagnostic message when constructing exception object

```
String errorMsg = "Content-Length:" + contentlen + ", Received: " + rcvlen;  
throw new EOFException(errorMsg);
```



Madhavan Mukund

Exceptions in Java

Programming Concepts using Java 6/12

सिद्धिर्भवति कर्मजा

- Example: you write a method `readData()`
 - Header line provides length of data
 - `Content-Length: 2048`
 - Actual data read is less than promised length
- Search Java documentation for suitable pre-defined exception
 - `EOFException`, subtype of `IOException`
 - "Signals that EOF has been reached unexpectedly during input"
- Create an object of exception type and **throw** it


```
throw new EOFException();
```
- Can also pass a diagnostic message when constructing exception object


```
String errorMsg = "Content-Length: " + contentlen + ", Received: " + rcvrlen;
throw new EOFException(errorMsg);
```



Madhavan Mukund

Exceptions in Java

Programming Concepts using Java 6/12

So, let us look at these checked exceptions. So, supposing you are trying to read some data from some source, and maybe one of the data items, the first line of the data item tells you something about how much data there is. So, this typically happens, for instance, when web browsers read web pages. So, there is usually a content header, as it is called, which tells you how many how long the web pages, how many bytes two web pages.

So, supposing the first line tells you that it should be 2048 bytes. And now you actually read the text and the, the file or the text ends before 2048 bytes. So, you get less than the promised length. Now, you could ignore this and go ahead or you could say that there is a mismatch. There is something that was promised and something that you got which was not correct. So, you want to do something about this. So, now you want to throw an error.

So, the first thing is what type of error to throw. So, the simplest thing is to try and throw an error, which is already present in the Java library of exceptions. So, if you search through the Java documentation, then you find that there is an exception called end of file exception EOF exception. And this is a subtype of this IO exception that we talked about in the previous example.

So, what does an EOF exception do according to the Java documentation, it signals that end of file has been reached unexpectedly during the input. Which is pretty much what has happened here because you were looking for 2048 bytes, and you unexpectedly discovered the end of the

input before that. So, this seems like a reasonable thing to do to throw this exception. So, now this is an exception class. So, remember that an exception itself is an object.

So, what you do is you create a new object of this time and throw it, you say throw, so throw is the command. And the argument that you pass the command is a new exception object, new EOF exception. So, of course, you could also do it separately, you could say EOF exception, you could create the this in it EOF exception is equal to new blah, blah, blah, and then you can say, throw E.

So, you can do it in two steps, to first create the exception object and then throw that created object or as in this case, you are implicitly creating and throwing an object anonymously, because you do not really need the name. We are just making an object of this type and then passing it back. So, this is the simplest form of constructor, which is available for all these exception objects inside Java, you can also provide this exception object with a message.

So, you might want to declare why you are creating this EOF exception. So, you might create a message which says, the content length was so much, and I received only so much. So, you can construct a string, which signifies the details of the error that you actually want to report. And then you can create this now new exception object by passing this error message as a parameter. So, when you create an exception object you can either create it with no parameters.

So, it just produces a kind of empty object which just says, this is what happened. And from the type of object, you can try to decide what error it was. Or you can provide a string as a diagnostic which tells you in addition to the type of object, what was the specific context in which this error was generated?

(Refer Slide Time: 11:46)



Throwing exceptions ...

- How does caller know that `readData()` generates `EOFException`?
- Declare exceptions thrown in header

```
String readData(Scanner in)
    throws EOFException {
    ...
    while (...) {
        if (!in.hasNext()) {
            // EOF encountered
            if (n < len) {
                String errmsg = ...
                throw new EOFException(errmsg);
            }
        }
        ...
    }
    return(s);
}
```



Madhavan Mukund

Exceptions in Java

Programming Concepts using Java 7/12



Throwing exceptions ...

- How does caller know that `readData()` generates `EOFException`?
- Declare exceptions thrown in header
- Can throw multiple types of exceptions

```
String readData(String filename)
    throws FileNotFoundException,
    EOFException { ... }
```

```
String readData(Scanner in)
    throws EOFException {
    ...
    while (...) {
        if (!in.hasNext()) {
            // EOF encountered
            if (n < len) {
                String errmsg = ...
                throw new EOFException(errmsg);
            }
        }
        ...
    }
    return(s);
}
```



Madhavan Mukund

Exceptions in Java

Programming Concepts using Java 7/12

सिद्धिर्भवति कर्मजा

- How does caller know that `readData()` generates `EOFException`?

- Declare exceptions thrown in header

- Can throw multiple types of exceptions

```
String readFile(String filename)
    throws FileNotFoundException,
        EOFException { ... }
```

- Can throw any subtype of declared exception type

```
String readFile(String filename)
    throws IOException { ... }
```

- Can throw `FileNotFoundException`, `EOFException`, both subclasses of `IOException`

```
String readData(Scanner in)
    throws EOFException {
    ...
    while (...) {
        if (!in.hasNext()) {
            // EOF encountered
            if (n < len) {
                String errmsg = ...
                throw new EOFException(errmsg);
            }
            ...
        }
        return(s);
    }
}
```



So, we said before, that it is okay to generate an exception inside a function, but how does the caller know about it. Because the caller also has to take action based on whether or not a function can provide an error back. If I call a function, and I am not aware that this function might generate an error, then I might not be able to take it. First of all, I may not be able to take suitable action because I do not know what kind of error is going to generate. But secondly, my code which I believe was robust might crash for reasons, which I do not know.

So, for many reasons, it is not good for a function to generate an exception without the other functions that use it being aware of it. So, what Java insists is that you must indicate in the header of the function, any exceptions that you explicitly throw. So, if in your code, you have this, that you throw a new EOF exception with this error message, because you got less than the expected amount of data.

Then you must say upfront that I am capable of throwing a new EOF exception as one of the valid behaviors of my code. So, if everything goes, well, I will give you back a string. But if things do not go, well, I might instead give you a new EOF exception. So, this has to be declared. Now, you can throw more than one type of exception, you do not have to just like we can catch more than one type of exception in one try statement.

You can also generate at different points, different things may go wrong. So, you might be reading from a file and the file may not be there, you might be trying to read the file and it might end up having fewer bytes and you are expected. So, you could throw for instance, both of

depending on the context of file not found exception or a new EOF exception. So, you can give a sequence of exception types in the header separated by commas to indicate that these are all the types of exceptions which I am capable of throwing.

Now, just like arguments are compatible with the hierarchy. So, is this header with throws. So, when I say throws IO exception, what I mean is it can throw IO exception. But it can also throw anything which is a subclass of IO exception. So, if I say that it throws IO exception, then the code that receives this exception should be prepared for something which is either an IO exception or a subclass of it.

So, if I can generate a file not found exception, I can generate an EOF exception because both of these are actually subclasses of IO exception. So, both while catching an error where I use the class hierarchy, and I can get a more specific error for a more generic argument type and when throwing a thing I can advertise a more generic error type and throw a more specific one. So, the class hierarchy is kind of the way that the class hierarchy interacts with arguments is uniform in this error handling and exception handling as it is when functions are called in normal things.

(Refer Slide Time: 14:37)

Throwing exceptions ...

- Method declares the exceptions it throws
- If you call such a method, you must handle it

```
String readData(Scanner in)
throws EOFException {
    while (...) {
        if (!in.hasNext()) {
            // EOF encountered
            if (n < len) {
                String errmsg = ...
                throw new EOFException(errmsg);
            }
            ...
        }
        return(s);
    }
}
```

Madhavan Mukund Exceptions in Java Programming Concepts using Java 8/12

Throwing exceptions ...

- Method declares the exceptions it throws
- If you call such a method, you must handle it
- ... or pass it on; your method should advertise that it throws the same exception

```
String readData(Scanner in)
    throws EOFException {
    ...
    while (...) {
        if (!in.hasNext()) {
            // EOF encountered
            if (n < len) {
                String errmsg = ...
                throw new EOFException(errmsg);
            }
            ...
        }
        return(s);
    }
}
```



Madhavan Mukund

Exceptions in Java

Programming Concepts using Java 8/12

Throwing exceptions ...

- Method declares the exceptions it throws
- If you call such a method, you must handle it
- ... or pass it on; your method should advertise that it throws the same exception
- Need not advertise unchecked exceptions
 - Error, RuntimeException

```
String readData(Scanner in)
    throws EOFException {
    ...
    while (...) {
        if (!in.hasNext()) {
            // EOF encountered
            if (n < len) {
                String errmsg = ...
                throw new EOFException(errmsg);
            }
            ...
        }
        return(s);
    }
}
```



Madhavan Mukund

Exceptions in Java

Programming Concepts using Java 8/12

सिद्धिर्भवति कर्मजा

- Method declares the exceptions it throws
- If you call such a method, you must handle it
- ... or pass it on; your method should advertise that it throws the same exception
- Need not advertise unchecked exceptions
 - `Error`, `RuntimeException`
- Should not normally generate `RuntimeException`
 - Fix the error or report suitable checked exception

```
String readData(Scanner in)
    throws EOFException {
    ...
    while (...) {
        if (!in.hasNext()) {
            // EOF encountered
            if (n < len) {
                String errmsg = ...
                throw new EOFException(errmsg);
            }
            ...
        }
        return(s);
    }
}
```



Madhavan Mukund

Exceptions in Java

Programming Concepts using Java 8/12

So, the method now has declared it. Now, the flip side of this is as you call the method, you must handle it. So, this is something now for the compiler. So, this is so, Java as we have seen before tries to make as many things visible at compile time as possible. So, as many inconsistencies as possible should be flagged at compile time and not when the code is running. So, if you call this function then you know that this function has this EOF exception as a possibility.

So, then it is your obligation to handle it. So, what does handling it mean? Handling it means that you must have a try block for it. If you do not have a try block what we said is that this function will now pass this error back to the calling function. So, implicitly your function also throws an EOF exception. So, you have two choices, I call this function read data, either I do a try read data and catch this exception and do something with it or I do not do a try.

But then it is possible because read data generated EOF exception that this code generates EOF exception. Then in my header, I must say that this is also possible that I throw an EOF exception because I sort of pick it up from a function that I am calling whose exception and not handling. So, this is now something that the Java compiler actually enforces. So, if you do not do one of these two things, either catch it, or declare it as a throw throwable exception from your function, then Java will say that there is a compilation problem with this function.

So, there are some sort of exceptions to say use a common word. So, there are some exceptions to this rule. So, in particular, if the code that you generate a code that you call creates an error or a runtime exception. Then as we said, these are things which are happening because of some

execution time problems. And it is not your responsibility to catch these. So, if the person who wrote the code was sloppy, and they did not check an array out of bounds you are not obliged to report.

So, if you are only calling checked exceptions, then you have to report it. So, you either catch it, or you have to report it by saying that it is also checked exception for my header. But if it is an error or a runtime exception. So, that is why these are called unchecked. So, Java has compiler does not actually check for these exceptions in the code. So, just to highlight, you should not normally generate this. So, it may be generated because of sloppy code.

The person forgot to check it and therefore, at runtime, the program crashes, and it generates a runtime exception on its own. But it is not good practice for you as a programmer to say, Oh, $i > n$. And therefore, I will generate a runtime exception. If you have detected that i is bigger than the length of the array, then you should provide a better exception than that. So, you should provide an checked exception, which explains what happened.

So, do not rely on untimed exceptions. If you I mean runtime exceptions, if you have a runtime exception that you are checking for, then please generate some exception, which is more meaningful than just the runtime exception. So, the calling code knows what to do with it. So, the calling code is aware that you are checking for this.

(Refer Slide Time: 17:53)

IIT Madras
BSc Degree

Customized exceptions

- Don't want negative numbers in a `LinearList`
- Define a new class extending `Exception`

```
public class NegativeException extends Exception {  
    private int error_value;  
    // Negative value that generated exception  
    public NegativeException(String message, int i) {  
        super(message); // Appeal to superclass  
        error_value = i; // constructor to set message  
    }  
  
    public int report_error_value() {  
        return error_value;  
    }  
}
```

Madhavan Mukund

Exceptions in Java

Programming Concepts using Java 9 / 12

- Don't want negative numbers in a `LinearList`
- Define a new class extending `Exception`
- Throw this from `LinearList`
 - Note that `add` advertises the fact that it throws a `NegativeException`

```
public class NegativeException extends Exception{
    ...
}

public class LinearList{
    ...
    public add(int i) throws NegativeException{
        ...
        if (i < 0){
            throw new NegativeException("Negative input");
        }
        ...
    }
}
```



Madhavan Mukund

Exceptions in Java

Programming Concepts using Java

9/12

So, checked exceptions typically arise in specialized circumstances as we said, when you have some code, which has some constraints, which are not part of the normal data structure constraint. So, we have a linear list for instance which stores some order numbers, and we do not want any negative numbers in this. Now, there is no negative non-negative integer type, which will generate an automatic runtime exceptions. So, we have to do something about this.

So, in order to flag this, we have to first create a customized exception class. So, we will perhaps call it negative exception. So, everything that we want to generate must sit under throwable. And in particular, it must sit under exception. Because remember, the throwable split says error and exception. So, if we extend the exception class then we get automatically all the default functions and all that which are defined in that hierarchy up to exception.

So, in this class, what we have said is that we will store the error, the offending error. So, this happens, because somebody tried to insert a negative number into our linear list, which does not like negative numbers. So, we will store that thing. So, when we construct an exception of this type, there are two things. One is the message that is to be printed to the user. And the other thing is this information about what negative number created.

So, what we can now do is, in order to get this error to actually incorporate this message, we just call the super type constructor. The constructor associated with our negative exception, and we put this message there. And then because we have this information here, we set this error value

to be minus 1 or not minus 1 to be i the value that you created. And now we might add a public function which reports this error value.

So, these are now because this class is going to be usable like any other class. So, if somebody gets this, this particular exception, then they can look at the public methods and say, Oh, this is the exception, I can actually find out what is the mistake by looking at this report error value function. And now when I define my linear list class, I would define in the add where I add an element to the linear list, I would define that.

When i is negative, it throws a new negative exception. So, this is the message that the error would generate negative input. And this information, if you remember, gets stored inside as that public private integer, which is exposed to that public method, which allows you to retrieve the offending field. Now, the other thing to note is that because I am now throwing this negative exception, I have to advertise it. So, therefore, this add function now has an extra component to its header, which says that it throws a negative exception.

(Refer Slide Time: 20:40)



More on catching exceptions

- Can extract information about the exception

```
try {
    ...
    call a function that may
    throw an exception
}
catch (ExceptionType e){
    ...
    String errorMsg = e.getMessage();
    ...
}
```



Madhavan Mukund Exceptions in Java Programming Concepts using Java 10 / 12

More on catching exceptions

- Can extract information about the exception
- Chaining exceptions
 - Process and throw a new exception from `catch`

```
try {
    ...
    access database
    ...
}
catch (SQLException e){
    ...
    String errorMsg =
        "database error" + e.getMessage();
    throw new ServletException(errorMsg);
    ...
}
```



Madhavan Mukund

Exceptions in Java

Programming Concepts using Java

10/12

More on catching exceptions

- Can extract information about the exception
- Chaining exceptions
 - Process and throw a new exception from `catch`
- `Throwable` has additional methods to track chain of exceptions
 - `getCause()`, `initCause()`

```
try {
    ...
    access database
    ...
}
catch (SQLException e)
{
    ...
    String errorMsg =
        "database error" + e.getMessage();
    throw new ServletException(errorMsg);
    ...
}
```



Madhavan Mukund

Exceptions in Java

Programming Concepts using Java

10/12

सिद्धिर्भवति कर्मजा

More on catching exceptions

- Can extract information about the exception
- Chaining exceptions
 - Process and throw a new exception from `catch`
- `Throwable` has additional methods to track chain of exceptions
 - `getCause()`, `initCause()`
- Add information when you chain exceptions

```
try {
    ...
    access database
}
catch (SQLException e){
    ...
    String errorMsg =
        "database error" + e.getMessage();
    ServletException newe =
        new ServletException(errorMsg);
    newe.initCause(e);
    throw newe;
}
```



Madhavan Mukund

Exceptions in Java

Programming Concepts using Java 10/12

More on catching exceptions

- Can extract information about the exception
- Chaining exceptions
 - Process and throw a new exception from `catch`
- `Throwable` has additional methods to track chain of exceptions
 - `getCause()`, `initCause()`
- Add information when you chain exceptions
- Retrieve information when you catch exception

```
try {
    ...
}
catch (SQLException e){
    ...
    Throwable original = e.getCause();
    ...
}
```

Original.getCause()



Madhavan Mukund

Exceptions in Java

Programming Concepts using Java 10/12

So, if I am now at the other side, and I get this negative exception, then I might want to find out about it. So, the most basic thing I can do is to get the error string associated with that message. So, this is defined in throwable itself. So, the highest level in this error hierarchy is throwable. So, it has some default function defined. So, one of them is get message. So, you can always extract the error message associated with an error by using this get message.

So, remember that you get this as an object. So, then you can invoke this and this is an exception type. So, it will always be compatible with a throwable type. Now, the other thing that you can do is to take an error of one type and pass it on as an error of another type. So, supposing you

have some code which is trying to access a database. Now, you have returned some kind of server application a servlet as it is called.

So, you get in your code some error from the database, which is of type SQL exception. So, what you want to say is that this SQL exception that I receive from the backend from the database, I am going to pass on as a servlet exception to the function that called there. So, I am going to throw a new servlet exception. And in this new servlet exception, I am going to pass on the message that I got from the database.

So, I add this string database error, and then I pass whatever message I got back. So, that the servlet exception carries in some sense some history about where it came from. So, this is why it is called chaining, I am taking one exception, and kind of passing it on with some extra information involved. Now, you can do a little bit better than this. So, you can actually do more than just pass on this message.

So, now, what we are saying is, we are passing on this message, but we do not have access to this `e` anymore. This `e` is the only thing that the code that we wrote here did was it extracted the message that `e` provided and stuck it into a new error. So, the new error only has that error message, it does not have any further diagnostic information that came from the SQL database. But logically, if I know that the error came from the SQL database, I might want to know that error as well not just the error that has been generated in this servlet.

I want to know more about the SQL exception. So, then throwable has this thing called `getCause` and `initCause`. So, what I can do now is I can make this a little bit more refined, I can first set up this error message, then I can create this new error object, which is a servlet exception error message. And now I can add some extra information to it through this `initCause` I can say that the `initCause` of this new error is this old error `e`. So, this should be throw new `e`. Sorry.

So, I got an error, `e` from which I built a new error new `e`. So, I record in new `e`, it is like a linked list in some sense of errors. So, I have an error, which is a servlet error, and it points through this `cause` thing to the error that generated so, I have hidden inside. So, this is this chaining so it is like a list or it is like an nested box. So, the box servlet error contains inside it the SQL error. So, any information I want about SQL error I can now extract.

Because on the other side I can get the servlet exception and it has an e it is the outer box I can get the inner box by calling the corresponding get cause. So, init cause was used to create the inner box now get cause extract that inner box. So, now the function that called the servlet can now look one level deeper into the SQL error and find out everything about it. So, this is this idea of chaining.

So, you can kind of nest the errors once inside the other end as you pass them up. So, at the top level you can go down as may be. So, in particular, if the SQL error itself had another error rate, then I can look at `original.getCause()`; and find out whether the SQL error itself was generated by one more error. Eventually when you get to the bottom most error as you would expect this get cause will be null.

So, when you find that the get causes null, you know that this was the original error in some sense. This was the source of all the errors, but till that you can keep following, it is like following links in a linked list, you can keep going get cause get cause get cause until you hit null. So, you get all the errors on the chain that generated this error. So, this is chaining errors.

(Refer Slide Time: 25:23)

Cleaning up resources

- When exception occurs, rest of the `try` block is skipped
- May need to do some clean up (close files, deallocate resources, ...)
- Add a block labelled `finally`

```
try{
    ...
} catch (ExceptionType1 e){ ... }
catch (ExceptionType2 e){ ... }
finally{
    ...
    // Always executed, whether try
    // terminates normally or
    // exceptionally. Use for clean up.
}
```

Madhavan Mukund Exceptions in Java Programming Concepts using Java 11/12

Cleaning up resources

- When exception occurs, rest of the `try` block is skipped
- May need to do some clean up (close files, deallocate resources, ...)
- Add a block labelled `finally`
- Different scenarios

```
FileInputStream in =
new FileInputStream(...);
try {
    // 1
    code that might throw exceptions
    // 2
}
catch (IOException e) {
    // 3
    show error message
    // 4
}
finally {
    // 5
    in.close();
}
// 6
```



Madhavan Mukund

Exceptions in Java

Programming Concepts using Java 11/12

Cleaning up resources

- When exception occurs, rest of the `try` block is skipped
- May need to do some clean up (close files, deallocate resources, ...)
- Add a block labelled `finally`
- Different scenarios
 - No error — 1,2,5,6

```
FileInputStream in =
new FileInputStream(...);
try {
    // 1
    code that might throw exceptions
    // 2
}
catch (IOException e) {
    // 3
    show error message
    // 4
}
finally {
    // 5
    in.close();
}
// 6
```



Madhavan Mukund

Exceptions in Java

Programming Concepts using Java 11/12

सिद्धिर्भवति कर्मजा

Cleaning up resources

- When exception occurs, rest of the `try` block is skipped
- May need to do some clean up (close files, deallocate resources, ...)
- Add a block labelled `finally`
- Different scenarios
 - No error — 1,2,5,6
 - `IOException` in `try`, no exception in `catch` — 1,3,4,5,6

```
FileInputStream in =
    new FileInputStream(...);
try {
    // 1
    code that might throw exceptions
    // 2
}
catch (IOException e) {
    // 3
    show error message
    // 4
}
finally {
    // 5
    in.close();
    // 6
}
```



Madhavan Mukund

Exceptions in Java

Programming Concepts using Java

11/12

Cleaning up resources

- When exception occurs, rest of the `try` block is skipped
- May need to do some clean up (close files, deallocate resources, ...)
- Add a block labelled `finally`
- Different scenarios
 - No error — 1,2,5,6
 - `IOException` in `try`, no exception in `catch` — 1,3,4,5,6
 - `IOException` in `try`, chained exception in `catch` — 1,3,5

```
FileInputStream in =
    new FileInputStream(...);
try {
    // 1
    code that might throw exceptions
    // 2
}
catch (IOException e) {
    // 3
    show error message
    // 4
}
finally {
    // 5
    in.close();
    // 6
}
```

exception



Madhavan Mukund

Exceptions in Java

Programming Concepts using Java

11/12

Another thing that we have to be careful about is what happens when we abort code due to a try. So, when an exception occurs, we said the rest of the try block is skipped. And this problem might arise that the rest of the try block might be doing something which you need to do like you might want to close a file. Or you might do something else you might want to delete some node in a list or something like that. Which you expect it to do, but it did not happen because your try block aborted in the middle.

So, what you can now do is add an extra special block at the end called finally. So, finally has the property that it will always be executed. So, it will be executed, whether or not you come

through whether the try terminates normally or it terminates exceptionally. So, there is no way to bypass the finally.

So, whichever way you come through this, either you come through here, or you come through here, or you come through here, all of these will eventually hit finally. So, just to understand exactly what it means. So, let us look at this more elaborate thing. So, we have here code, which initially opens some file input stream. And finally suppose you close it. So, it tries something with this file input stream which might throw some exceptions.

And one of them is an IO exception. And in this finally block, now we have put this close. So, we do not want to put the close here because it might get skipped. So, I put the finally block here. So, what are the different things that could happen. So, the first thing that can happen of course is there is no error. So, if there is no error, then I come through this block unscathed, and at this point, I go over to the finally. So, I do 1, 2, then I do 5, and then I do 6.

So, 6 is the code after this. So, basically, the one of the properties of the try block is it does not abort as such unless there is propagated uncaught. So, if anything happens inside and the error is caught it will go to the next statement. So, 6 will happen after this try block is finished, but it will execute 1, 2 and 5, 1 and 2 from the try block and 5 from the initial it from the finally. So, this is what happens if there is no error.

Now, the next possibility is that there will be an error here, which will take it here. But this actually goes through fully with no further error being propagated. So, then what happens is that I do 1, and then I hit an error in that code, which might throw an exception. So, that throws me to 3. But 3 goes after to the end of its thing without doing anything. And then from here, I come to the finally. So, then I do 5 and 6. So, I do 1, 3, 5, and 6.

So, the only thing that gets skipped is this 2, which was in the try. But there is a third possibility. The third possibility is that in this itself, some error happens, which I did not anticipate. So, this might be an error, which takes me I might throw another exception, for instance. So, you might think that when you throw an exception inside a catch that you get out of it immediately but you do not.

So, supposing you have a chain exception, like we saw earlier SQL error decided to throw a servlet error. Even then you will come into the finally. So, if I have say for instance, somewhere

here, if I chain an error, then I will do 1, and I will come here, then I will do 3. And then when I show an error message, I might simultaneously throw a new chain error message. But I will still come to 5. But now because I have thrown an exception.

I am not going to continue with this code anymore. Because I have effectively said go away. So, I throw an exception. In the other two cases, I handle the exception. So, I proceeded to the 6th block. Here I have thrown an exception, but before I go away, I will still execute the finally here. So, the finally will always be executed whether or not you continue after the finally depends on whether you throw another exception or not. So, I hope that clarifies exactly what happens with the finally block.

(Refer Slide Time: 29:28)

IIT Madras
BSc Degree

Summary

- Use **try-catch** to safely call functions that may generate errors
- Can **throw** an exception — usually checked exception
- Must advertise checked exceptions that are thrown in function header
 - Java compiler enforces that code that calls such a function handles the exception or passes it on
- Can inspect exceptions and chain them with information about original source
- Use **finally** to clean up resources that may be left open when code is interrupted by an exception

Madhavan Mukund Exceptions in Java Programming Concepts using Java 12/12

So, to summarize we have this try catch mechanism, which is there also in Python to safely call a function that may generate errors. So, you try to execute the code. And you can use multiple catch statements to catch the different types of errors that the code may throw. And on the other side, you can throw an exception, which is usually a checked exception. So, if you detect something anomalous when the code is running, you can freely throw it.

But when you throw it, you must advertise it. So, Java insists that code was generated exceptions must have it in the header. And this serves two purposes. One is from a kind of socially useful point of view, which is the programmers who call your function must be aware of this. They should not accidentally crash their code because you crash the code. But the other thing is that

Java actually now uses this to compare the code that is calling your code with the code that you have defined.

So, Java has compiler will enforce that the code which calls such a function, which throws a checked exception, actually either handles that exception or explicitly passes it back as a checked exception of its own. So, Java actually does a compile time check of compatibility between calls and definitions of functions that throw exceptions. So, exceptions are complex objects. So, we can look at them and get information about them and we can put information into them also.

So, we define we saw that negative exception in which we stored as a private variable, the negative number which generated the exception, which you could then extract through a public function like a get function which caught it or we could have this get cause kind of thing which gives us a chain of exceptions. So, we can therefore use this exception object structure in order to encode a fairly rich amount of information about this exception and whatever triggered it.

So, that the code that gets the exception back is able to examine this and take appropriate action. And in order to deal with this problem of code, which should have got executed but did not typically to do with resource allocation files being closed and all that. We have this extra finally block which comes at the end of a try, which is always executed no matter how the try end whether the try is normal whether exception happens. The exception is handled properly the exception throws a chain exception. All these cases the finally will definitely be executed.